Modelling Deadlock Resolution in Open Restricted Queueing Networks

Geraint I. Palmer-Liyu

October 16, 2026

1 Introduction

2 Preliminaries and System Model

This work builds directly on the work of [2], which considered deadlock detection in discrete event simulations (DES) of open restricted queueing networks, and analysed Markovian models of deadlocking queueing system. The simulation software used is the Ciw library in Python [3], although the methodology is appropriate for general DES. We are concerned with open restricted queueing networks that experience Type I blocking. Queueing networks are described as open if customers can enter and leave the system from the exterior, while restricted networks contain at least one service centre with limited queueing space or capacity before it, turning away externally arriving customers when full, and inducing blocking for internally routed customers when full.

This paper considers Type I blocking only [1]: once a customer finishes their service and wishes to join another node's queue, they are blocked from joining that queue if it is full; while blocked that customer remains with its server until capacity becomes available at its destination; during this time that server is unavailable to begin another customer's service This gives rise to the idea that one customer can be blocked by another customer: that is if customer A at node 1 is blocked from entering node 2 because it is full, then customer A is waiting on the customers who are with the servers at node 2 to exit, hence customer A is blocked by the customers who are with the servers of node 2. *Deadlock occurs when a subset of blocked customers are blocked by customers in that subset only.*

In [2] a graph theoretic approach to detecting deadlock during a simulation run is given: Let $D(t) = (V, E(t))$ be the state digraph of a queueing system, with set of vertices V corresponding to the servers of all service centres, and a set of directed edges $E(t)$ representing blockage relationships: if a customer at the kth server of node i is blocked from entering node j, then there are directed edges from the vertex corresponding to node i's kth server to every vertex corresponding to the servers of node j. Then it is shown that *deadlock occurs at time t if and only if $D(t)$ contains a knot*, that is a strongly connected component with no sink. Figure 1 shows an example of a three node queueing network in deadlock with the corresponding state digraph.

This paper builds on this graph theoretic foundation to consider deadlock resolutions with no perceptible disruption to the customers' or the servers' experiences. First let's consider the unblocking behaviours, that is the cascading sequence of servers freeing customers and capacity freeing up that occurs when a customer blocking other customers is able to free a server. By understanding this behaviour we then design an deadlock resolution method in Section 4.

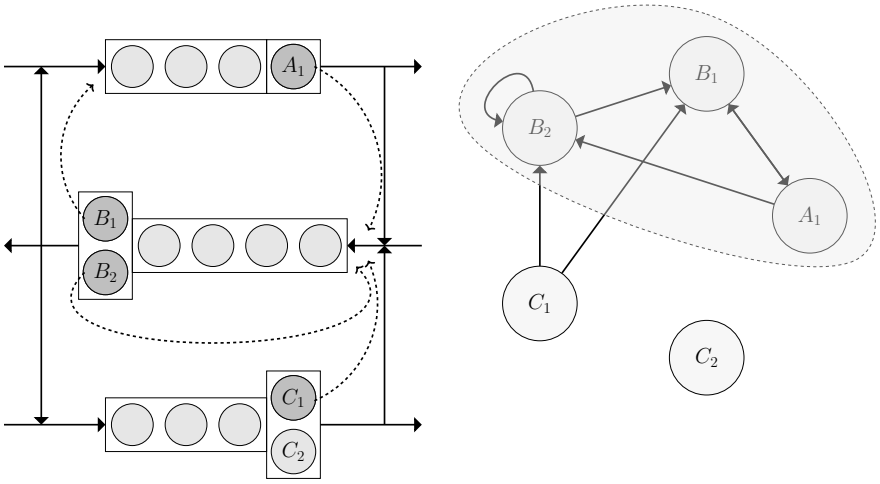


Figure 1: Example of a three node network in deadlock, and the state digraph $D(t)$ that describes the blocking relationships. Customers with servers A_1, B_2 and C_1 are blocked to node B, while the customer with server B_1 is blocked to node A. The knot within $D(t)$ is highlighted - note that server C_1 is not part of the knot, as no deadlocked customer is waiting for this server to be free.

3 Sequences of Unblockages

At any time t during the simulation, some subset of the customers may be blocked. Define a blockage event as one specific instance of a specific customer being blocked, and write B_k as the kth blockage to occur during the simulation run. Now:

Definition 1. *Let $(B_n)_{n\in\mathcal{N}(t)}$ be the sequence of active blockage events at time t, where $\mathcal{N}(t) \subset \mathbb{N}$, and $|\mathcal{N}|$ is bounded above by the number of servers in the queueing network.*

Blockage events are associated with both individual customers and the servers who are occupied with the blocked customers. Recall that V, the set of vertices of the state digraph $D(t)$, represents the set of all servers, therefore there is a mapping between blockage events and servers:

Definition 2. *Let $v : (B_n)_{n\in\mathcal{N}(t)} \to V$ be an injective mapping between blockage events and servers, such that $v(B_i) \in V$ represents the server associated with blockage event B_i.*

Note that $v(\cdot)$ is injective as each blockage event must be associated with a server, and each server can only be with one customer at a time, and so can only be associated with one blockage event at a time. $v(\cdot)$ is not surjective, as not all servers necessarily have to contain blocked customers.

Let's consider how the system evolves over time t. Both the set of active blockages $\mathcal{N}(t)$ and the sequence $(B_n)_{n\in\mathcal{N}(t)}$ evolve with t. Let's also introduce $M(t)$, the total number of blockages that have occurred so far in the simulation run. Then with each blockage and each unblockage the system state is updated simultaneously by:

- When a customer at server $v^\star$ is blocked to node i with servers $V_i = \{v_j \in V \mid v_j \text{ is a server of node } i\}$:
  - The total blockage counter $M(t)$ increments by 1,
  - The active blockage set $\mathcal{N}(t)$ expands to $\mathcal{N}(t) \cup \{M(t) + 1\}$,
  - Create $B_{M(t)+1}$ such that $v(B_{M(t)+1}) = v^\star$,

  - The set of edges of the state digraph $E(t)$ includes the new blockage relationships to become $E(t) \cup \{(v^\star, v_j) \mid v_j \in V_i\}$.

- When the blockage event B_k becomes unblocked:
  - the index leaves the active blockage set $\mathcal{N}(t)$ to become $\mathcal{N}(t) \setminus \{k\}$,
  - The associated edges are removed from the state digraph, so $E(t)$ becomes $E(t) \setminus \{(v(B_k), v_j) \mid v_j \in V\}$.

Table 1 shows how a sequence of blockages and unblockages can lead to the deadlock seen in Figure 1, and how $(B_n)_{n\in\mathcal{N}(t)}$, $\mathcal{N}(t)$, $M(t)$, and $D(t)$ evolve at each step. In this example there are three nodes: Node A with one server A_1, Node B with two servers B_1 and B_2, and Node C with two servers C_1 and C_2.

An unblockage can itself trigger further unblockages, for example if there exists a set of blockages $\{B_i, B_j, ...\}$ which are all blocked by $v(B_k)$, then unblocking B_k will allow one of $\{B_i, B_j, ...\}$ to become unblocked. This choice is governed by two conditions given in Remark 1.

Remark 1. *Unblocking B_k will also trigger the unblocking of B_h if:*

1. *$(v(B_h), v(B_k)) \in E(t)$, that is that the edge between the server associated with B_h and the server associated with B_k in the state digraph; or in other words, that B_h is blocked by B_k; and*
2. *$h = \min(i \in \mathcal{N}(t) \mid (v(B_i), v(B_k)) \in E(t))$, that is out of all the active blockages caused by B_k, B_h was the first to become blocked.*

Note that condition 1 is a necessity, an unblockage can only occur when there is spare capacity at the destination node, which is triggered by the original unblockage. Condition 2 meanwhile is a choice: it would be natural in a FIFO service discipline that those customers who have waited longest should be unblocked first, retaining the eventual queueing order at the destination node; though in other service disciplines other selection criteria may be more appropriate. As an example, Figure 2 shows a system with a sequence of blockages, and Table 2 shows the sequence of unblockages that would occur *instantaneously* when server B_1 releases their customer and frees capacity at Node B. This sequence of cascading unblockages eventually unblocks servers C_1, B_2 and A_1, while server C_2 remains blocked.

To illustrate the arbitrariness of condition 2 above, consider the case where server A_1 was the first to get blocked. Once server B_1 releases their customer and creates capacity at Node B, A_1 is unblocked, but as there are no other blockages caused by A_1, condition 1 is not satisfied, and so there are no more unblockages. Hence servers B_2, C_1 and C_2 all remain blocked.

TODO

Show that this is the true protocol for Ciw and SIMUL8. Do this (in an Appendix) by designing a deterministic system where the sequence of unblockages does indeed change macro behaviour: show we get the same results in Ciw as we do in SIMUL8.

4 Deadlock Resolution

4.1 Induced Sequence of Unblockages

For a cascading sequence of unblockages, which 'untangles' the blockages built up over time, to occur, it is necessary for the customer at the head of the sequence to be blocked by servers who have the potential to create capacity at their node. That is, blocked by servers that are themselves are not blocked, or that the system is not deadlocked.

We introduce the idea of artificially *inducing* a sequence of unblockages by creating dummy capacity, triggering a cascading sequence of unblockages. The aim is that this will clear the deadlocked state seamlessly, that

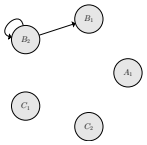
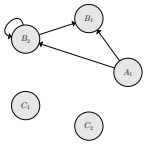
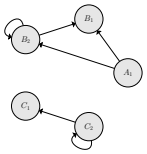
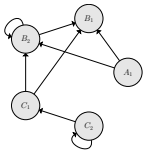
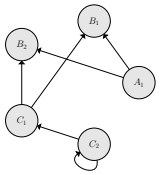
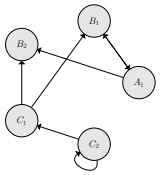
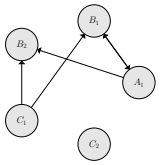
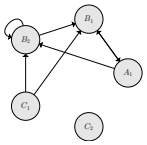
Event	$M(t)$	$\mathcal{N}(t)$	$(B_n)_{n\in\mathcal{N}(t)}$	$D(t)$
Customer at B_2 is blocked to Node B	1	$\{1\}$	B_1	
Customer at A_1 is blocked to Node B	2	$\{1,2\}$	B_1, B_2	
Customer at C_2 is blocked to Node C	3	$\{1,2,3\}$	B_1, B_2, B_3	
Customer at C_1 is blocked to Node B	4	$\{1,2,3,4\}$	B_1, B_2, B_3, B_4	
Customer at B_2 is unblocked	4	$\{2,3,4\}$	B_2, B_3, B_4	
Customer at B_1 is blocked to Node A	5	$\{2,3,4,5\}$	B_2, B_3, B_4, B_5	
Customer at C_2 is unblocked	5	$\{2,4,5\}$	B_2, B_4, B_5	
Customer at B_2 is blocked to Node B	6	$\{2,4,5,6\}$	B_2, B_4, B_5, B_6	

Table 1: An example sequence of blockages with unblockages.

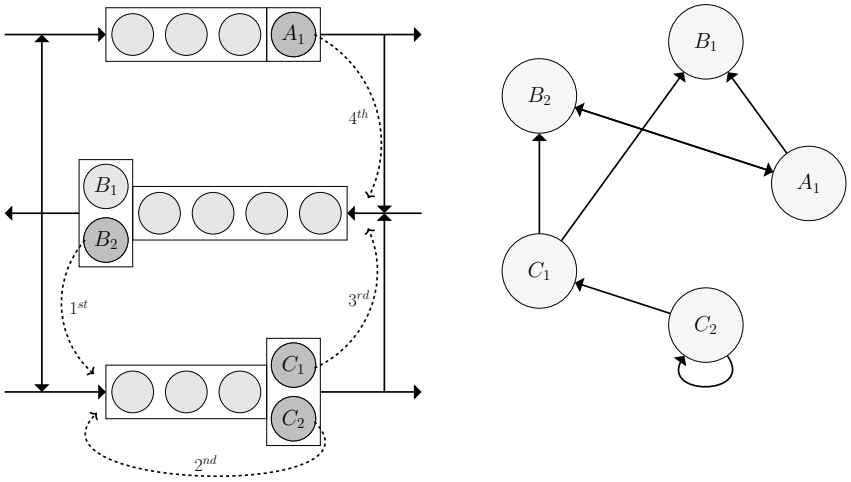


Figure 2: Example of a sequence of blockages and corresponding state digraph $D(t)$, for which Table 2 shows the sequence of unblockages when the customer at server B_2 exits the system.

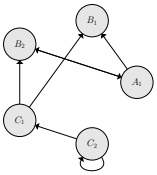
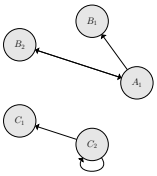
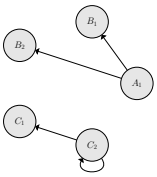
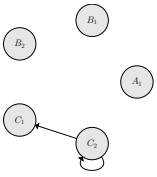
Event	$M(t)$	$\mathcal{N}(t)$	$(B_n)_{n\in\mathcal{N}(t)}$	$D(t)$
Customer at B_1 is released	4	$\{1,2,3,4\}$	B_1, B_2, B_3, B_4	
Customer at C_1 is unblocked to Node B	4	$\{1,2,4\}$	B_1, B_2, B_4	
Customer at B_2 is unblocked to Node C	4	$\{2,4\}$	B_2, B_4	
Customer at A_1 is unblocked to Node B	4	$\{2\}$	B_2	

Table 2: An example sequence of cascading unblockages.

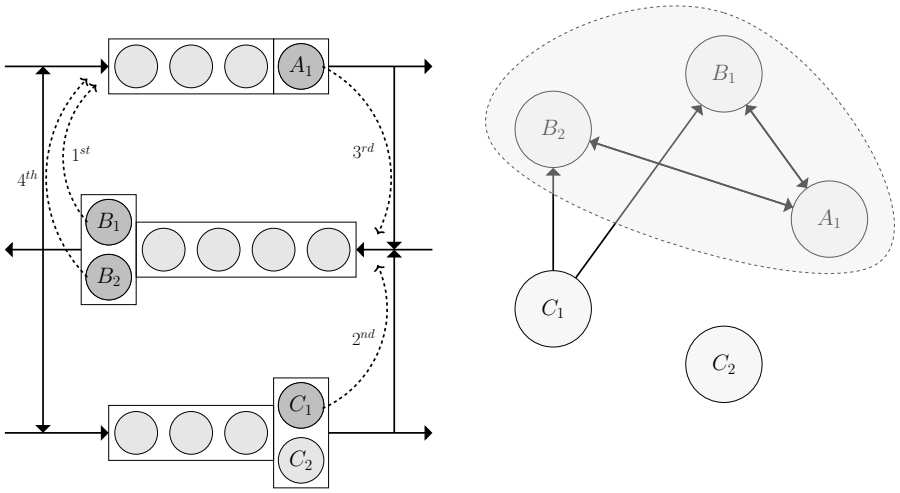


Figure 3: Example of a deadlock where the induced sequence of unblockages by adding dummy capacity at Node A would not be feasible under the conditions set out in Remark 1.

is without breaking the queueing capacity of Type 1 blocking rules for any meaningful duration of simulation time.

Definition 3. *At deadlock, a sequence of unblockages is* INDUCED *by temporarily creating extra capacity for one customer at one of the nodes.*

The induced sequence of unblockages is FEASIBLE *if by the end of the sequence that extra capacity is not in use, and can therefore be removed.*

As sequences of unblockages happen instantaneously with respect to simulation time, a feasible induced sequence of unblockages will not cause any customer to experience any non-zero duration of time at the dummy capacity, and the nodes themselves will not experience any non-zero duration of time with extra capacity. Therefore this feasible induced sequence of unblockages plays no role in the modelled system, and is purely a simulation mechanic that stands as a proxy for mutual communication between deadlocked service centres.

Remark 2. *Not all induced subsequences are feasible under the conditions set out in Remark 1.*

The claim in Remark 2 can be shown with a simple counter-example: consider the three node queueing network in deadlock given in Figure 3, and consider inducing a sequence of unblockages by adding dummy capacity to Node A. Then customer at B_1 will become unblocked, filling the dummy capacity and leaving capacity at Node B; which in turn unblocks the customer at C_1, filling the capacity at Node B and leaving capacity at Node C. However no customer was originally blocked by C_1, and so no more unblockages can occur, and the dummy capacity introduced at Node A is still used. Hence this is not a feasible induced sequence of unblockages.

5 Markov Models of Deadlock Resolution

References

[1] Raif O Onvural and HG Perros. On equivalencies of blocking mechanisms in queueing networks with blocking. *Operations Research Letters*, 5(6):293–297, 1986.

[2] Geraint I Palmer, Paul R Harper, and Vincent A Knight. Modelling deadlock in open restricted queueing networks. *European Journal of Operational Research*, 266(2):609–621, 2018.

[3] Geraint I Palmer, Vincent A Knight, Paul R Harper, and Asyl L Hawa. Ciw: An open-source discrete event simulation library. *Journal of Simulation*, 13(1):68–82, 2019.